Async / Await Programming in Python

A beginner-friendly explanation with FastAPI examples

Main idea: async/await is mainly about not wasting time while a program is waiting for I/O such as databases, APIs, files, or network responses.

1. Synchronous code

Normal Python functions are usually synchronous. If an operation takes time, the function waits until that operation finishes before continuing.

```
def get_data():
    data = download_from_api()   # takes 3 seconds
    return data
```

During those 3 seconds, this function is essentially saying: “I am waiting. Do not continue until I get the result.”

2. What does async mean?

Using **async def** defines an asynchronous function (a coroutine function). It allows the function to cooperate with Python's asynchronous event loop.

```
async def get_data():
    ...
```

3. What does await mean?

await pauses the current coroutine until an asynchronous operation completes. While it is waiting, the event loop can let other asynchronous tasks make progress.

```
import asyncio

async def get_data():
    print("Getting data...")
    await asyncio.sleep(3)
    print("Got data!")

asyncio.run(get_data())
```

The important point is that **await asyncio.sleep(3)** does not needlessly block the event loop for three seconds. Other async tasks can run during that waiting period.

4. Why is this useful?

Web servers often spend time waiting: one request may wait for a database, another for an external API, and another for a network response. Async programming lets the event loop work on other requests during those waits.

```
User 1 -> database -> WAIT ----------> result
                      |
User 2 --------> database -> WAIT ----> result
                               |
User 3 -------------> API -> WAIT ----> result
```

5. Simple restaurant analogy

Synchronous: Order food → stand at the counter doing nothing → food is ready → continue with something else.

Asynchronous: Order food → food is cooking → do something else → food is ready → return to it.

A useful mental model for **await** is: “I need to wait for this operation, so let other async work run in the meantime.”

6. Why you see async/await in FastAPI

FastAPI supports asynchronous route functions, which are useful when the route calls async-compatible databases, HTTP clients, or other I/O operations.

```
@app.get("/users")
async def get_users():
    users = await database.get_users()
    return users
```

Here, **async def** defines an asynchronous route and **await database.get_users()** waits for the database operation while allowing the event loop to handle other async work.

7. When async is useful

Async is especially useful for **I/O-bound** work: database queries, HTTP/API requests, network operations, WebSockets, and async file operations.

It is not automatically a way to make heavy CPU calculations faster. CPU-heavy work may require other approaches such as multiprocessing, worker processes, or task queues.

Quick memory guide

async def = define asynchronous work
await = wait for async work without unnecessarily blocking other async tasks
event loop = coordinates which async task can make progress while others are waiting